LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title

CU6051NI Artificial Intelligence

75% Individual Coursework

Submission: Final Submission

Academic Semester: Autumn Semester 2025

Credit: 15 credit semester long module

Student Name: Kriti Bista

London Met ID: 23049271

College ID: NP05CP4A230021

Assignment Due Date: 21/01/2026.

Assignment Submission Date: 20/01/2026

Submitted To: Mr.Nikesh Regmi

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarde

Table of Contents

1 Introduction 1
1.1 AI 1
1.2 AI in our daily life 3
1.3 Explanation of the topic/AI concepts used 3
1.3.1 Recommendation System 3
1.4 Explanation/Introduction of the chosen problem domain/topic 4
2 Background 5
2.1 Research work done on the chosen topic/problem domain 5
2.1.1 Job Recommendation System 5
2.1.2 Research done on Context Based Filtering 5
2.1.3 Research done on collaborative filtering 5
2.1.4 Research done on dataset 6
2.2 Review and analysis of existing work in the problem domain 7
2.2.1 Research on job recommendation system using LinkedIn user Profiles 7
2.2.2 Research on Job recommendation system using machine learning 8
2.2.3 Research on job recommendation based on job profile clustering and job Seeker Behavior 10
3 Solution 12
3.1 Explanation of the proposed solution/approach to solving the problem 12
3.2 Dataset background 13
3.3 Explanation of the AI algorithm/algorithms used 14
3.3.1 Context based filtering 14
3.3.2 NLP collaboration 16
3.3.3 K- means clustering 18
4 pseudocode and Flowchart 21

4.1 Pseudocode of the NLP-Classification 21
4.2 Flow chart of NLP classification 24
4.3 Pseudocode of the content base filtering 25
4.4 Flow chart of content-based filtering 27
4.5 Pseudocode of k-clustering 28
4.6 Flowchart of K-means clustering 30
5 Explanation of development process 31
5.1 Software used 31
5.1.1 Jupyter notebook 31
5.2 Libraries used 32
5.2.1 Pandas 32
5.2.2 NumPy 33
5.2.3 Matplotlib 33
5.2.4 Seaborn 35
5.2.5 Scikit-learn 36
5.3 Components of library used 36
5.3.1 Cosine Similarity 36
5.3.2 TF-IDF Vectorizer 36
5.3.3 Logistic Regression 37
5.3.4 Principal Component Analysis (PCA) 37
5.3.5 K-Means Clustering 37
5.4 Development Process 37
5.4.1 Importing Required Libraries 37
5.4.2 Data Loading 38
5.4.3 Dataset Exploration 39
5.4.4 Data cleaning and Text Preprocessing 40
5.4.5 AI model Training 41

5.4.6 Content-Based job recommendation 42
5.4.7 NLP-Based job Category Classification 45
5.4.8 Job clustering using K-means 48
5.4.9 Cluster Visualization Using PCA 49
5.5 Model Evaluation and comparison 51
5.5.1 Comparison of AI Models used In Job Recommendation System 52
5.5.2 Comparison of Implemented AI Models 52
6 Conclusion 53
6.1 Analysis of the work done 53
6.2 How this addresses real world problems 54
6.3 Further work 54
7 References 55

Table of figures

Figure 1: AI 1
Figure 2: Recommendation system 4
Figure 3: User based and Items based 6
Figure 4: Research on Job recommendation system using LinkedIn user profile 7
Figure 5: Job recommendation system using machine learning 8
Figure 6: Job recommendation based on job profile clustering and job seeker behaviors 10
Figure 7: Term Frequency 14
Figure 8: Inverse documentation Frequency 15
Figure 9: TF-IDF vectorization algorithm 15
Figure 10: Dot product formula used in cosine similarity calculation. 15
Figure 11:Cosine similarity 16
Figure 12: K-means clustering with data points and cluster centers 19
Figure 13: K-means clustering showing data points and their cluster centroids 19
Figure 14:K-means clustering result with centroids and grouped data points 20
Figure 15: NLP classification Flowchart 24
Figure 16: Content based filtering flowchart 27
Figure 17: K-mean Clustering 30
Figure 18: Jupyter 31
Figure 19: Pandas 32
Figure 20: NumPy 33
Figure 21: Matplotlib 34
Figure 22:Seaborn 35
Figure 23: sklearn 36
Figure 24: Importing Libraries 37
Figure 25: Dataset Loading 38
Figure 26: Data Exploration 39
Figure 27: Data Cleaning and pre processing 40
Figure 28: Importing Dataset After Cleaning 41
Figure 29: Content-Based Filtering 42
Figure 30:Tf formula 43
Figure 31: TD-IDF Score calculation 43

Figure 32:Cosine Similarity 43
Figure 33: NLP classification 45
Figure 34: Logistic Regression Linear Function 46
Figure 35: Softmax Probability Function 47
Figure 36: Job role Prediction rule 47
Figure 37: K-mean Clustering 48
Figure 38:K-means Objective Function 48
Figure 39: Cluster Visualization 50
Figure 40: AI model comparison 51

Table of Tables

Table 1: Comparison of AI Models used In Job Recommendation System 52
Table 2: Comparison of Implemented AI Models 52

1 Introduction

1.1 AI

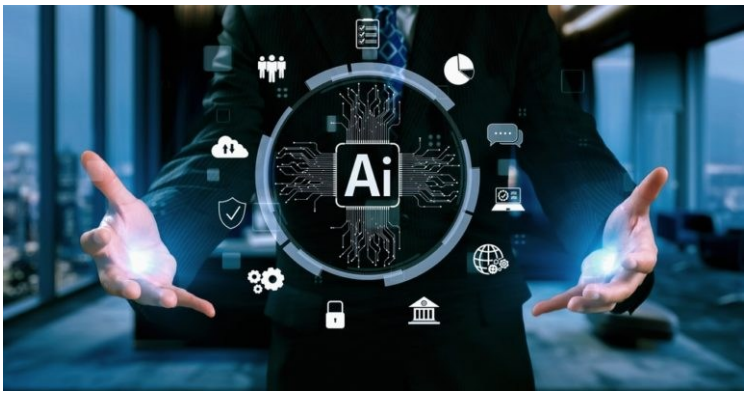


Figure 1: AI

Artificial intelligence (AI) can be defined as machine capability to carry out functions, which are typically characterized by human intelligence, and computer systems. Such tasks involve learning through data, logical reasoning, and performance through time. AI are used in expert systems, natural language processing (NLP), and speech recognition, computer vision and modern AI tools, like ChatGPT and Perplexity. A mass of the company's doing their products marketing using AI tools and techniques which has gained immense popularity and expanding at a very high rate. The technology under use is not always latest sometimes it relies on machine learning techniques which has been used around from long time. Special hardware and software are required to design and train machine learning models in order to develop AI systems. None of the programming languages is specifically used for AI. Nevertheless, Python, R, Java, C++, and Julia are common among AI developers because they allow processing of data and machine learning. (Craig, et al., 2024)

A core concept used in AI are:

1. **Machine learning**:

   Machine learning is a part of artificial intelligence, which assists systems to learn through data and discover hidden patterns. It allows algorithms to make predictions or decisions on new data, without writing code for each. Applications of machine learning include image and speech recognition, food and speech recognition, recommendation models, financial fraud, portfolio optimization, and robots. (GeeksforGeeks, 2018).There are many types of machine learning such as:

   - Supervised Learning
   - Unsupervised Learning
   - Reinforcement Learning

2. **Natural Language Processing**

   Natural Language Processing (NLP) is a branch of artificial intelligence, which is concerned with how computers interact with human language. Its biggest aim is to allow computers to process vast quantities of text and language data. NLP assists machines to comprehend, interpret and produce human language in a valuable and significant manner. Organizations such as open AI that have created language models such as ChatGPT have reiterated the importance of NLP in creating intelligent systems through which users can input information, provide the right output, and generate text, and therefore technology has become easier and more accessible to users (Crabtree, 2023).

3. **Data mining**

   Data mining is also known as Knowledge Discovery in Data (KDD) is a subfield of data science that involves the integration of computer software, machine learning, and statistics way to identified valuable information in extremely large data set. Machine learning assists computers to make their own learning without involving human beings directly. A larger part of data is created through online transactions with businesses, government offices, and education institutions. This data is referred to as a big data that is too large and complex

to be analyzed manually and therefore, computers are applied to accomplish the task. Data mining transforms this raw data into valuable knowledge that can assist organizations know more about their users or customers and provide answers to valuable questions. The typical applications of the data mining are in the examination of consumer behavior, forecasting patterns on sales, and fraud detection. (Nikolopoulou., 2023)

1.2 AI in our daily life

Artificial Intelligence is also very popular in our everyday life as it helps to simplify our choices, make them quicker, and more individual. AI is applied in phones in voice assistants, face recognition and intelligent typing suggestions. Google, YouTube, and Netflix online platforms are AIs that suggest content depending on user interests. AI in online shopping and job portals proposes products or jobs, which are relevant to the user. The other applications of AI are in navigation apps to identify the most optimal path, in healthcare to assist in diagnosis and in banking to identify fraud. overall, AI contributes to efficiency, precision, and user experience in the daily tasks.

1.3 Explanation of the topic/AI concepts used

1.3.1 Recommendation System

Recommendation systems are artificial intelligence-based programs which process user likes, preferences, and behavior to give personalized recommendations. It is Mostly popular in the e-commerce market, streaming applications, social media, and so on, they are used to improve the user experience by providing suggestions of products, movies, music, or any content that the user may be interested in. These systems use algorithms to suggest the interests of users and assist them to explore new products as well as increasing the activities and satisfaction to businesses. Recommendation systems are customized to various requirements with the help of collaborative, content-based, and hybrid. (Kumar, 2021)

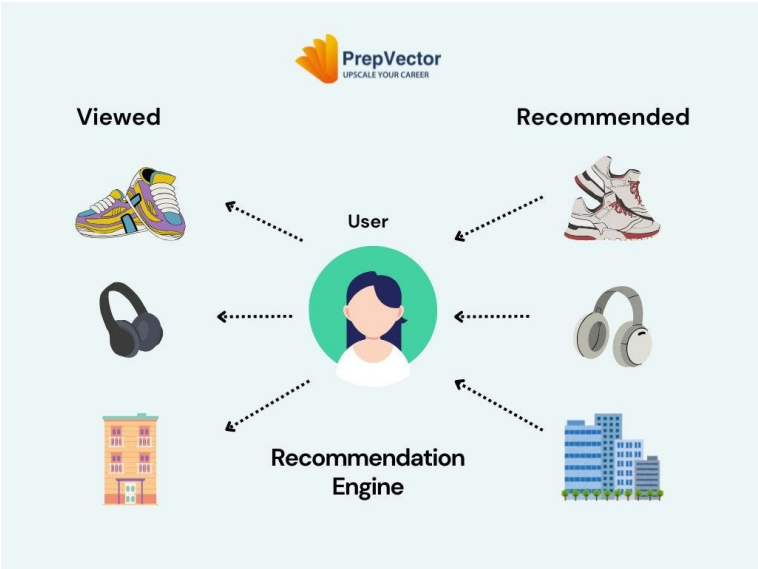


Figure 2: Recommendation system

1.4 Explanation/Introduction of the chosen problem domain/topic

The recruitment and job-matching space has changed in the current digital world at a rapid rate as a result of tremendous expansion of online job portals. Platforms daily create thousands of job ad, applications, and skill profiles, which have created a huge amount of unstructured information that users need to process. This increase in volume has posed a problem referred to as information overload where job seekers and employers are having a hard time trying to sift through the data and find and filter the right applicant respectively. Research has indicated that the volume and the pace of the digital data have rendered the customary search mechanisms by key-word inadequate to do successful job matching

In order to address such issues, data mining, machine learning, and recommender-system technologies have become a more important component of modern recruitment systems. Data mining can assist in identifying relevant patterns in large data sets, whereas the work of recommender systems is to alleviate the information overload in offering targeted job recommendations to the users in response to their skills, experience, and preferences. The Information Journal of MDPI indicates that recommender systems are important in converting massive and complex data into valuable, relevant, and useful

insights, which can be useful in decision making processes. These smart systems enhance employee satisfaction with the job search and efficiency on the part of the employers as job-matching is automated and optimized. (Alfaifi, 2024)

2 Background

2.1 Research work done on the chosen topic/problem domain

2.1.1 Job Recommendation System

A Job recommendation system refers to the smart technology that recommends the right job opportunities to the users depending on their skills, work experience, preferences, and previous application history. Such systems process different types of data, such as user profiles, resumes, job descriptions, and patterns of interaction to produce personalized job proposals that are very close to the career objectives of the user.

2.1.2 Research done on Context Based Filtering

Content-based filtering is an information retrieval technique that chooses and returns items related to a user's query based on item features. This approach frequently considers characteristics of other things in which a user shows interest. (Kavlakoglu, 2024)

This AI method suggests jobs by matching job descriptions and text of user profile. The system can use NLP algorithms like TF-IDF (Term Frequency-Inverse Document Frequency) and cosine similarity to determine the similarity between a job posting and the user in terms of the user and their skills and keywords. It is dedicated to the customized suggestions using only the personal data of the user.

2.1.3 Research done on collaborative filtering

Collaborative filtering refers to a process of retrieving information which suggests information to the users according to the interaction of similar users with the same preferences and behavior. That is, collaborative filtering algorithms classify users according to their behavior and provide general group properties to suggest items to a target user. The principle of collaboration

recommender systems works under the assumption that like users (in terms of their behavior) have similar interests and similar tastes. (Ph.D, 2024)

There are two main ways it is used:

1. **User-Based Collaborative Filtering**:
   The method is used to find users with like preferences like that of the target user and recommend products that some other similar users have liked.
2. **Based Collaborative Filtering:**
   This process identifies products that have similar characteristics with the ones that the intended user has been interested in and then recommends the same to the user.

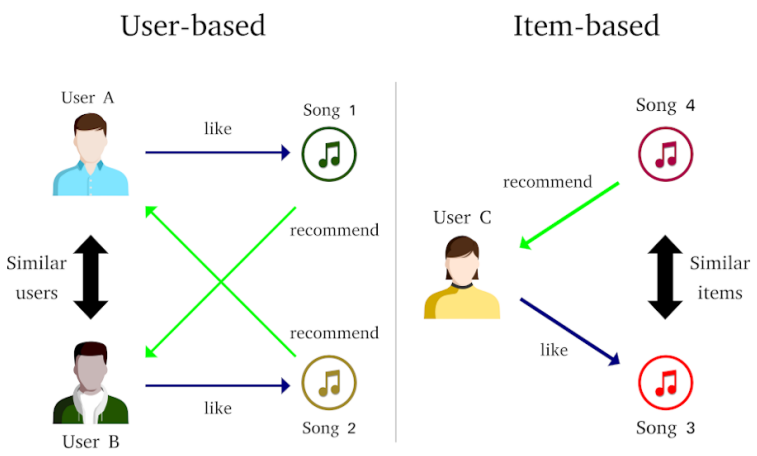


Figure 3: User based and Items based

2.1.4 Research done on dataset

A dataset is a group of structured information, which is arranged and kept collectively to be analyzed or processed. Data in a dataset are generally interconnected somehow and are obtained out of one source or towards one project. As an example, a dataset may include a set of business information sales rates, contact details of buyers, purchases. There are a wide variety of different types of data that can be included in a dataset, such as numerical data or text, audio, or image recordings. The information contained in a dataset may be accessed individually, collectively or as a single unit. The data analytics, data analysis and machine learning (ML) are impossible without datasets that

serve as the foundation on which a data analyst can base the insights and trends. They are critical to ML since one of the most important steps to begin the successful training and implementation of an ML model is to choose the appropriate dataset to use in that project. (databricks, 2023)

2.2 Review and analysis of existing work in the problem domain

2.2.1 Research on job recommendation system using LinkedIn user Profiles

Figure 4: Research on Job recommendation system using LinkedIn user profile

The study on “Job Recommendation System using LinkedIn User Profiles” focuses on how job matching in professional networking platforms, like LinkedIn, can be enhanced with the help of intelligent recommender systems. The research study aims at proposing the right jobs to individuals and the right employees to the employers in handling the problems of information overload and possible discrimination in the recruitment systems. The job and user data are large and diverse; thus, the authors highlight the necessity of learning-based models of recommendation compared to traditional rule-based methods.

The research study explores two primary recommendation methods, which include content-based filtering and collaborative filtering. Content-based filtering are based on the analysis of job description and user profile through feature representation methods (N-grams, trigrams, engineered features). The collaborative filtering algorithm predicts job preferences based on user behavior

using a matrix factorization model, where the user and job vectors are dot product similar, and optimized based on gradient descent. Content-based recommendations were also trained using a neural network-based model and Singular Value Decomposition (SVD) was used to reduce the dimension and enhance training.

The proposed methods were evaluated by using real-world job recommendation dataset sourced from Kaggle and performance of the system was measured by precision and recall metrics. The findings indicated that collaborative filtering registered a precision of between 0.53 to 0.80 and a recall of between 0.42 to 0.70 whereas the content-based filtering registered great precision of between 0.68 to 0.88 and a recall of between 0.60 to 0.84. The most successful model was a combination of content-based filtering with trigram features and Random Forest algorithm which proved more effective and accurate. (Haldar, 2024)

Overall, the research demonstrates that job recommendations systems based on machine learning can be effective to enhance job matching on professional platforms. The content-based filtering will give better and personalized recommendation as compared to collaborative filtering. Such techniques are beneficial in minimizing the information overload and enhancing the entire recruitment process.

2.2.2 Research on Job recommendation system using machine learning

Figure 5: Job recommendation system using machine learning

The Study shows “Job Recommendation System using Machine Learning “offers a smart job recommendation system to minimize the gap between job-seekers and recruiters. The study will deal with the problem of information overload due to the growing number of job applicants and job ads. The key aim of the research was not only to suggest the candidates the appropriate jobs, depending on their abilities and preferences, but also to assist the recruiters in finding the most suitable candidates in a cost-effective manner.

The system was implemented based on content-based filtering method which is aimed to match user preferences with job characteristics. machine learning model was developed using Python and executed through Jupyter Notebooks. The first stage involved cleaning and pre-processing of job and user data after which feature extraction and selection was done. To classify job descriptions into existing job roles like Jet Developer and App Developer, Natural Language Processing (NLP) methods were used using SpaCy library. then using feature overlap, these job categories were then compared with user inputs to determine similarity. The recommendation process was based on calculating similarity by dot product operations which make, the system scalable and independent of rest users’ data, hence avoiding the cold-start problem.

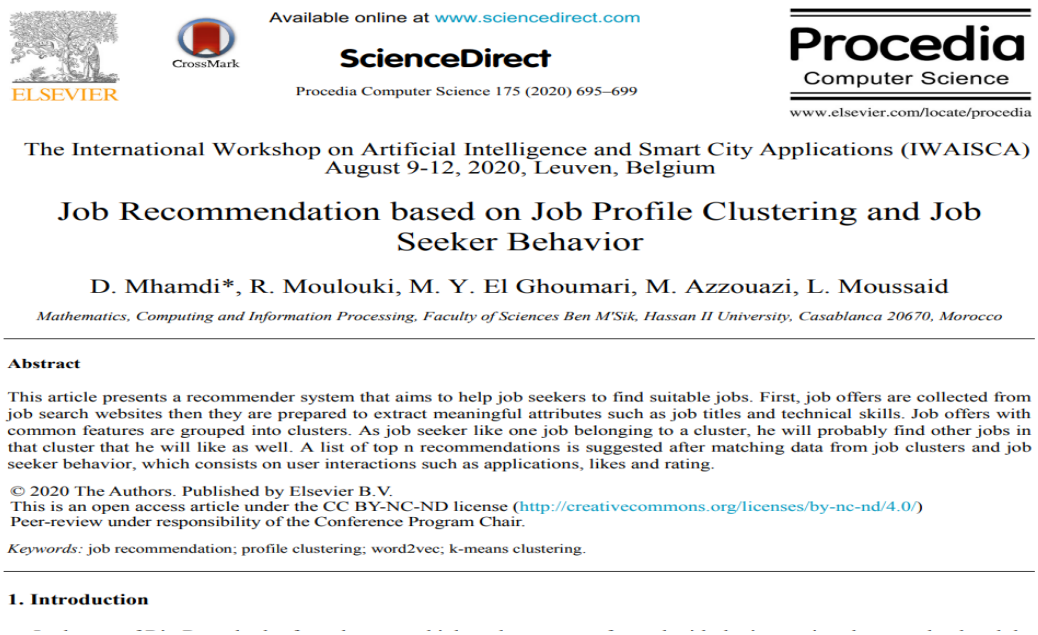
For supporting the recommendation system dataset from the U.S. Technology Jobs which is in Dice.com and the Stack Overflow Developer Survey Were used. These databases offered information related to job advert, skills, technology, and preference of developers., The outcomes of machine learning provided to users using Android application developed using Kotlin and Jetpack Compose according to the principles of Material 3 design. Communication between the android application and the FastAPI based python server was done using a Ktor backend server whereas data storage was done using MongoDB Atlas. Access control was through the JWT tokens to have a secure access.

The study evaluated system performance by standard evaluation metrics including precision and recall. The results revealed that content-based filtering method proved to be efficient in producing job suggestions that were relevant

as it matched the user skills with job requirement. The system successfully simplified the job-hunting a simpler task, as it gave job-seekers a personalized job recommendation and allowed users to explore relevant job opportunities easily. (Gadegaonkar, et al., 2023).

Overall, the study highlights effectiveness of machine learning-based content-based filtering systems are an efficient approach to enhancing the accuracy of job recommendations and optimizing the process of recruitment suggest system will make the job-hunting process easier and simplified, will enhance the user experience, and will demonstrate the great potential in the further development, supporting recruiters, improving machine learning models, and increasing the scale and recommendation rates.

2.2.3 Research on job recommendation based on job profile clustering and job Seeker Behavior



ELSEVIER

CrossMark

Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 175 (2020) 695–699

Procedia Computer Science

www.elsevier.com/locate/procedia

The International Workshop on Artificial Intelligence and Smart City Applications (IWAISCA)
August 9-12, 2020, Leuven, Belgium

Job Recommendation based on Job Profile Clustering and Job Seeker Behavior

D. Mhamdi*, R. Moulouki, M. Y. El Ghoumari, M. Azzouazi, L. Moussaid

Mathematics, Computing and Information Processing, Faculty of Sciences Ben M'Sik, Hassan II University, Casablanca 20670, Morocco

Abstract

This article presents a recommender system that aims to help job seekers to find suitable jobs. First, job offers are collected from job search websites then they are prepared to extract meaningful attributes such as job titles and technical skills. Job offers with common features are grouped into clusters. As job seeker like one job belonging to a cluster, he will probably find other jobs in that cluster that he will like as well. A list of top n recommendations is suggested after matching data from job clusters and job seeker behavior, which consists on user interactions such as applications, likes and rating.

© 2020 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)
Peer-review under responsibility of the Conference Program Chair.

Keywords: job recommendation; profile clustering; word2vec; k-means clustering.

1. Introduction

Figure 6: Job recommendation based on job profile clustering and job seeker behaviors

Over the past few years, job portals online have become extremely rapid generating huge numbers of job-related information. Due to this fact, job seekers tend to have a hard time finding jobs that fit their skills, interests, and preferences. Meanwhile, recruiters are also finding it difficult to recruit individuals with the appropriate qualifications and skills. This distance make renders the process of recruitment ineffective and slow.

Job recommendation systems have been proposed as smart machine learning to address this issue. The systems are beneficial because they make recommendations to the users on the type of job to seek based on job requirements and user behavior. Job information obtained on various job websites is processed to obtain valuable information about the job like job titles, skills and technologies required. similarly, relevant job advertisements are clustered to allow users to receive more relevant suggestions.The activities of the users like job applications, likes, and rating can also be taken into consideration to know the interests of the users. Moreover, unstructured job descriptions and candidate profiles are analyzed with the help of Natural Language Processing (NLP). Applying machine learning and text analysis, job recommendation systems allow both job seekers and recruiters to work less, to save time, and give more precise and individual recommendations about jobs.

The study reveals that automated recommendation systems may be used to assist the users in discovering the items that are relevant to them, based on their preferences and actions.
The important type of recommendation system were identified:

- **Content based filtering**: This suggests those jobs that are related to the ones that a user had previously viewed or liked.
- **Collaborative filtering**: This suggests jobs by analyzing the choice of jobs by choices of other users sharing the same interests or preferences.
- The proposed system will cluster job offers based on similar characteristics and suggest them based on matching job offer clusters with user actions like likes and ratings.

- Text clustering techniques, Such as Word2Vec and K-means clustering proved to be useful in clustering similar job postings according to job titles and descriptions.

Overall, the paper presented a job recommendation model, which is concerned with the extraction of useful information about job postings through the text clustering method. Jobs are also clustered into groups according to similarities and appropriate jobs suggested to users according to the analysis of their past interaction. To improve the model in future, Word2Vec and k-means clustering can be used to analyze job description and user preference. This will assist in aligning job seekers to the job listings that are relevant. Data used in this system will be scrapped out of internet job portals. (Mhamdi, et al., 2020)

3 Solution

3.1 Explanation of the proposed solution/approach to solving the problem

Jobs that are posted on the internet are also many, and searching through them can be so time-consuming. Thus, job recommender systems are important in that they should provide very precise predictions that meet the preferences of the users. In our endeavor, we have adopted a content-based recommendation approach that would offer the appropriate job opportunities to job seekers. The system accepts user inputs such as skills, experience and qualifications and uses the content-based recommendation technique. This is done through the application of the cosine similarity and Term Frequency-Inverse Document Frequency techniques. One of the established Natural Language Processing techniques, Term Frequency-Inverse Document Frequency is used to transform text into an array of various vectors. Later the similarity of two vectors in terms of cosine similarity is employed to measure the similarity of vectors as well as measuring the similarity of words. This will lead to improvements in accuracy, quality, and scalability and at the same time, the memory consumption is minimal. (Saini & Tripathi, 2024)

3.2 Dataset background

Dataset used in Job Recommendation System is from Hugging Face. An open and popular platform where different types and sizes of dataset are available. The dataset choose for the project is a huge sample of actual job ads obtained through the Internet job boards. It includes information that is descriptive and detailed like job titles, job descriptions, name of the company, a job location, working schedule, remote working opportunity, and the range of salaries. The data is also clear and well sorted out containing extracted skills and standardized job categories that enhance ease of analysis. This dataset fits my project as it contains detailed textual job descriptions and well-organized job characteristics that are critical in the application of content-based filtering. Also, the volume of job records is quite high, which facilitates collaborative filtering since the pattern of similarity and trend between job postings can be discovered by the system. In general, the data set provides the high quality and real-life information that makes my Job Recommendation System more effective and accurate

The data is a secondary dataset because it was not gathered by the researcher. Instead, it was compiled, clean and which was released by the third party based on the data collected on the available job portals. The most obvious reason to choose secondary datasets because they are used in academic projects due to it is large-scale, reliable, and applicable in real-life scenarios.

Dataset Description

- The dataset contains a huge collection of Job posting related data.
- The dataset include position such as Data Analyst, Data Scientist and Data engineering
- The dataset contains both type of structured and un structured data.
- This dataset represents real-world job market data that is related to job seeker.
- Dataset contain schedule type like part time, full time, and salary information like yearly or hourly.

3.3 Explanation of the AI algorithm/algorithms used

3.3.1 Context based filtering

Content-based filtering is a recommendation system that comes up with highly personalized recommendations to individual customers. To illustrate, in the case of a user who likes watching the action movies, it is expected that he/she will be suggested to watch other movies the it will be action, instead of a romantic movie. The mechanism behind this method is prescribing the items that a user liked or has used previously. The difference between content-based recommendation systems and other types is that they do not rely on the information of the other users. Each of the recommendations is developed based on the information and preferences of the individual user only. (Stratascratch.com, 2023)

For job recommendation, Content Based Filtering using TF-IDF and Cosine similarity was chosen. This method was selected since the data is mainly in form of text of the jobs like job titles, skills, and descriptions. Content-based filtering is does not need historical data of user-job interaction, and makes the recommendation transparent and explainable by comparing user skills and interests are directly to job needs.

- Term Frequency shows how many times several words appear in a document. If the word occurs many times that word is considered as important for that document. The more often a word is used, the higher its TF value.

$$\mathrm{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{number of words in } d}$$

Figure 7: Term Frequency

- Inverse document frequency shows the important of word by analyzing how commonly or rarely it appear in all document. In case of TF it treats

all word same whereas, IDF only give priority to rare word or word which is used less important to very common words, Words which appear less get higher IDF values.

$$\mathrm{IDF}(t) = \log\left(\frac{N}{\mathrm{DF}(t)+1}\right)$$

Figure 8: Inverse documentation Frequency

- TF-IDE means Term Frequency-Inverse Document Frequency. This method is used to find how important a word is in a document as compared to many documents. It combines how often a word appears in one document with how rare that word is across all documents. (Kumari, 2024)

$$\text{TF-IDF}(t,d) = \mathrm{TF}(t,d) \times \log\left(\frac{N}{\mathrm{DF}(t)+1}\right)$$

Figure 9: TF-IDF vectorization algorithm

- How Cosine Similarity works

Cosine similarity measure by checking the angle between two vector in multi-dimensional space.It uses euclidean formula to calulcate the cosine of the angle between two vector that are not zero.

$$\mathbf{A} \cdot \mathbf{B} = \|\mathbf{A}\| \, \|\mathbf{B}\| \cos\theta$$

Figure 10: Dot product formula used in cosine similarity calculation.

- Cosine similarity applies when comparing two vectors A and B which have numerous values and it compares their similarity based on the dot product and the length.

$$\text{cosine similarity} = S_C(A, B) := \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \cdot \sqrt{\sum_{i=1}^{n} B_i^2}}$$

Figure 11:Cosine similarity

3.3.2 NLP collaboration

Natural Language Processing (NLP) is a branch of artificial intelligence, which concerns itself with the way in which computers understand the regular language used by people. It aims at facilitating computers to process and recognize huge volumes of data related to human language. NLP enables machines to comprehend, comprehend and produce language in an applicable and significant manner. The use of NLP to develop AI-powered systems that can interpret user input and generate responses, as well as generate writing is a popular focus of organizations like OpenAI that have developed language models like ChatGPT that further simplify and make technology easier and more accessible to users. (Crabtree, 2023)

- **How NLP works**

1. **Text preprocessing**

Text preprocessing is a key process in the NLP that preconditions machine learning with raw text. It is used in cleaning and standardization of text to eliminate noise and inconsistencies. Such typical processes are tokenization, that splits text into smaller components say words or sentences and converting all text to lowercase so that it is uniform, stopping words (which do not carry

significant information and lemmatization reducing words to their root form to reduce the number of words. Punctuation, special characters, and numbers are also eliminated in text cleaning. The preprocessing removes all dirt in the text and prepares it to be usable in feature extraction.

2. **Features Extraction**

The feature extraction transforms preprocessed text into numbers to enable the analysis of text using machine learning models. Other techniques like Bag of Words are used to represent text in terms of word frequencies whereas TF-IDF uses importance of words based on their frequency in a document relative to the whole body of data. More built-in models are word embeddings (Word2Vec and GloVe) that encode semantic interactions between words, and contextual ones which encode word meaning in context. These techniques will convert unstructured text into structured numerically predictable features.

3. **Text Analysis**

Text analysis deals with the knowledge and acquisition of rich information in the form of text data. It encompasses part-of-speech tagging that should determine grammatical roles, named entity recognition that should recognize significant structures in the sentence such as names and places, and dependency parsing that should understand the structure of a sentence. Sentiment analysis and topic modeling are other techniques that determine the tone of emotion and discern hidden topics in literature respectively. Natural Language Understanding (NLU) is also used to make text data meaningful and contextually aware to transform and convert it into helpful insights.

4. **Model Training**

Once the preprocessing and features extraction are complete, the trained machine learning models are then trained with the processed data. The model is trained to learn patterns through parameter modifications that make it minimize the errors and to achieve good performance. After being trained, the model is then able to make predictions on new, unseen data and refining of the model is achieved by evaluation, validation, and fine-tuning. This is assisted with several software tools. NLTK is a Python library and it has text processing

capabilities like tokenization, stemming, tagging, parsing and classification. TensorFlow is an open-source project, which is employed to train and optimize machine learning and NLP models with high efficiency.

3.3.3 K- means clustering

K-means clustering refers to an unsupervised machine learning technique that is used to classify data points in clusters without the help of labeled data. It one of the most applied clustering methods of machine learning. As the data is not labeled the algorithm is not based on some predefined categories or classes. There are various methods of clustering like overlapping, hierarchical and probabilistic methods are existing. k-means is category of exclusive or hard clustering in which each data value can belong to a single cluster only. The method is used in the market segmentation, document clustering, image segmentation and image compression data science operations. K-means is popular because it is easy to use, computationally effective, and the performance of the clustering is effective. (Kavlakoglu & Winland, 2024)

- **How it works**

K-means is a clustering algorithm that groups data into fixed number of clusters which is chosen in advance. Each cluster have its center point which is known as centroid, this are the average of all data points in that cluster. The algorithm keeps each data point into the cluster whose centroid is nearest, usually using Euclidean distance. The word means in k-means refers to finding this average value. The main aim of the algorithm is to keep data points within same cluster as close as possible, which automatically makes different clusters separate from each other. (NVIDIA Data Science Glossary, 2025)

- **In practice how its work:**

The k -means algorithm starts with initializing by selecting fixed number of clusters called K. After that keeps those K initial cluster centers, which can be done randomly or according to provide input values. These centers act as starting points for forming the cluster.

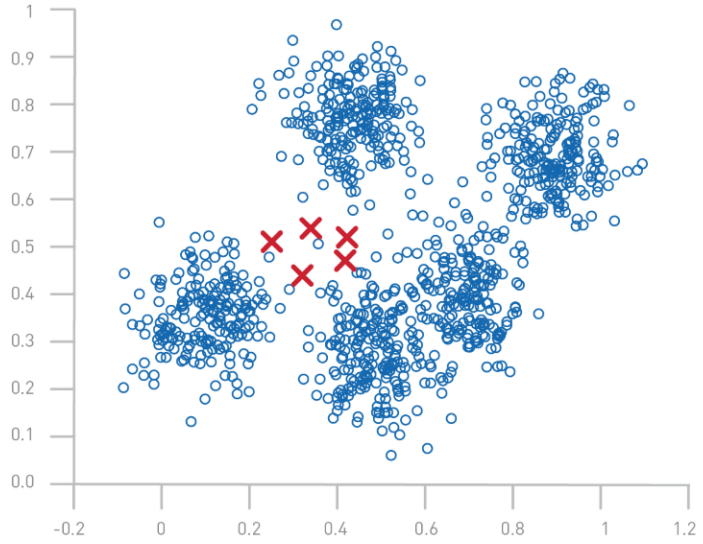


Figure 12: K-means clustering with data points and cluster centers

- In each pass of the algorithm, each point will be classified into the nearest cluster center.

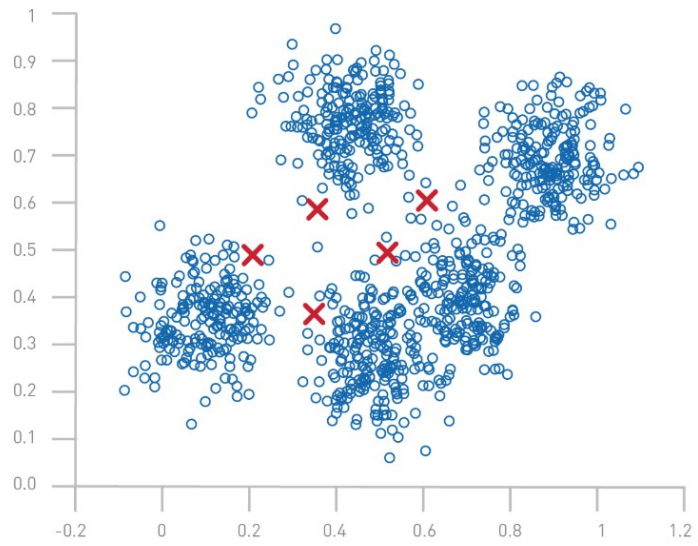


Figure 13: K-means clustering showing data points and their cluster centroids

- After the data points are assigned to clusters, the cluster centers are updated. Each center is shift to the middle of all the points in its cluster. This process is repeated and again until the cluster centers change very little or stop changing.

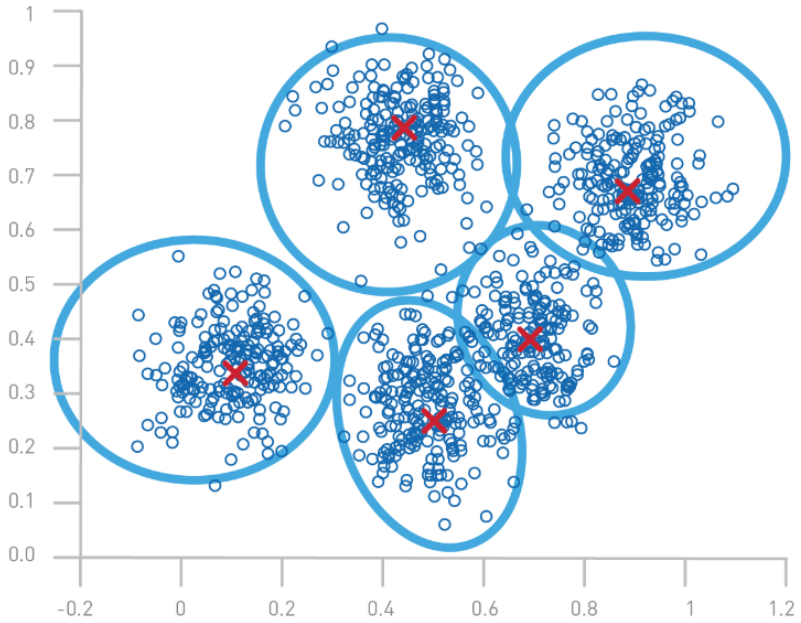


Figure 14:K-means clustering result with centroids and grouped data points

4 pseudocode and Flowchart

Pseudocode is a description of the how program or an algorithm in a simple readable language. It uses simple language and straightforward structure in such a way that the programmers can easily understand the logic of the program as well as other team members. Pseudocode is not a computer program that can be executed on a computer. It is rather a guide to assist in translating the concept of the program into a real programming language in the future. (Sheldon, 2023)

4.1 Pseudocode of the NLP-Classification

```
START

CLASS NLPClassificationModel:

 FUNCTION Load_Data():

    IMPORT (pandas, sklearn, TF-IDF, Logistic Regression) were needed.

    LOAD the job dataset

    STORE dataset as jobs

    RETURN jobs

 FUNCTION Prepare_Features(jobs):

    GET TF-IDF job vectors as feature set (X)

    GET job_title_short as target labels (y)

    RETURN X, y

 FUNCTION Split_Data(X, y):

    DIVIDE X and y into training and testing set.

    USE 80% Training and 20% testing  data respectively.

    RETURN Xtrain, Xtest, ytrain,ytest.

 FUNCTION Train_Model (X train, y train):

    IMPLEMENT Logistic Regression classifier

   ASSIGN Maximum iteration to achieve convergence.
```

```
    TRAIN the model using X train and y train to train
    RETURN trained_model
FUNCTION Build_User_Profile(user_skills):
    CONCAT user skills into one text string.
    TRANSFORM user competences to TF-IDF vector
    RETURN user_feature_vector
FUNCTION Predict_Job_Role(trained model, user feature-vector):
    RECOMMED the most appropriate type of job.
    RETURN predicted_job_role
FUNCTION Predict_Probabilities(trained-model, feature-vector of the user):
    EVALUTE the probability scores of each job category.
    RECORD  categories and probabilities into a table.
    SORT OUTPUT of probability In desending order.
    RETURN prediction_table
LOADS Loads the dataset with the job using Load_Data()
STORE the result as jobs
X,y = Prepare_Features(jobs)
X_train, X_test, y_train, y_test =
    Split_Data(X, y)
trained_model =
    Train_Model(X_train, y_train)
user_feature_vector =
    Build_User_Profile(user_skills)
```

```
 predicted_job_role =
    Predict_Job_Role(trained_model, user_feature_vector)
 prediction_table =
    Predict_Probabilities(trained_model, user_feature_vector)
 SHOW predicted_job_role
 SHOW prediction_table
END
```

4.2 Flow chart of NLP classification

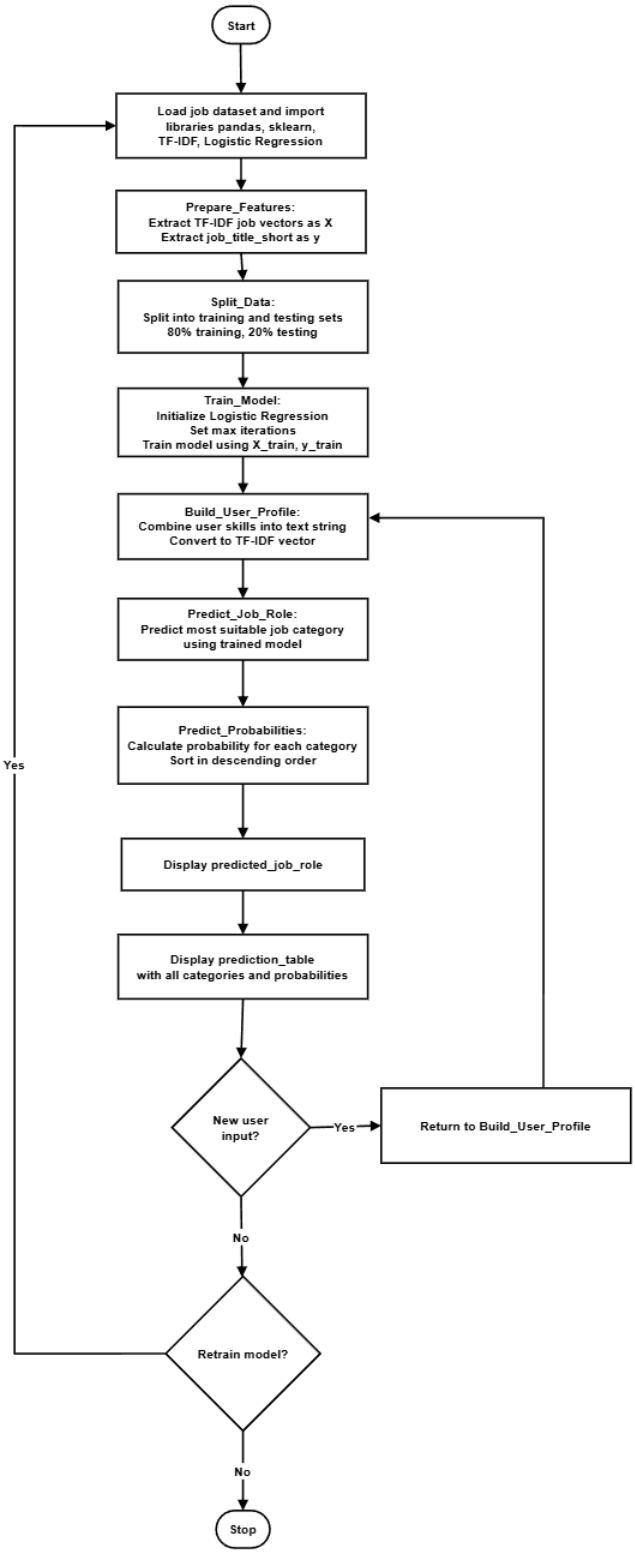


Figure 15: NLP classification Flowchart

4.3 Pseudocode of the content base filtering

```
START

CLASS ContentBasedFiltering:

 FUNCTION Load_Data():

    IMPORT necessary libraries (pandas, sklearn, TF-IDF, cosine similarity).

    LOAD the job dataset

    STORE dataset as jobs

    RETURN jobs

 FUNCTION Preprocess_Data(jobs):

    DELETE duplicate records or redundant records.

    HANDLE missing value in job title job skills, and job description.

    MERGE job title, job skills and job description in one text field.

    CHANGE text to lower-case.

    REMOVE r non informative characters (symbols, numbers, more spaces)

    RETURN preprocessed_jobs

 FUNCTION Build_User_Profile(user_skills):

    JOIN all the skills availed by users into a single text file.

    RETURN user_profile_text

 FUNCTION Feature_ Extraction(processed jobs, user profile text):

   ADD TF-IDF Vectorizer

IMPLEMENT TF-IDF Job text to create job feature matrix (tf-idf).

    CHANGE its user profile text using the same TF-IDF model.

    CREATE user_feature_vector

    RETURN job matrix, user feature vector
```

FUNCTION Cosine Similarity(job feature matrix, user feature vector): Similarity of user specific feature and job specific feature

CALCULATE_cosine between user feature vector and job features matrix.

STORE results as similarity scores.

RETURN similarity_scores

FUNCTION Get_Recommendations(similarity-scores, jobs, top-n):

SORT similarity scores in descending order

SELECT the top n jobs based on the maximum score of similarities.

RETURN recommended_jobs

LOAD the data set of the job with Load_Data()

STORE the result as jobs

processed jobs = Process_Data(jobs)

user_profile_text =

Build User Profile text (User Profile.)

job_feature_matrix,user_feature_vector=

Feature_Extraction(preprocessed_jobs, user_profile_text)

similarity_scores=

Compute_Cosine_Similarity(job_feature_matrix,user_feature_vector)

recommended_jobs =

similarity_scores, jobs, top n= 10)

Find Recommendations

DISPLAY recommended_jobs

END

4.4 Flow chart of content-based filtering

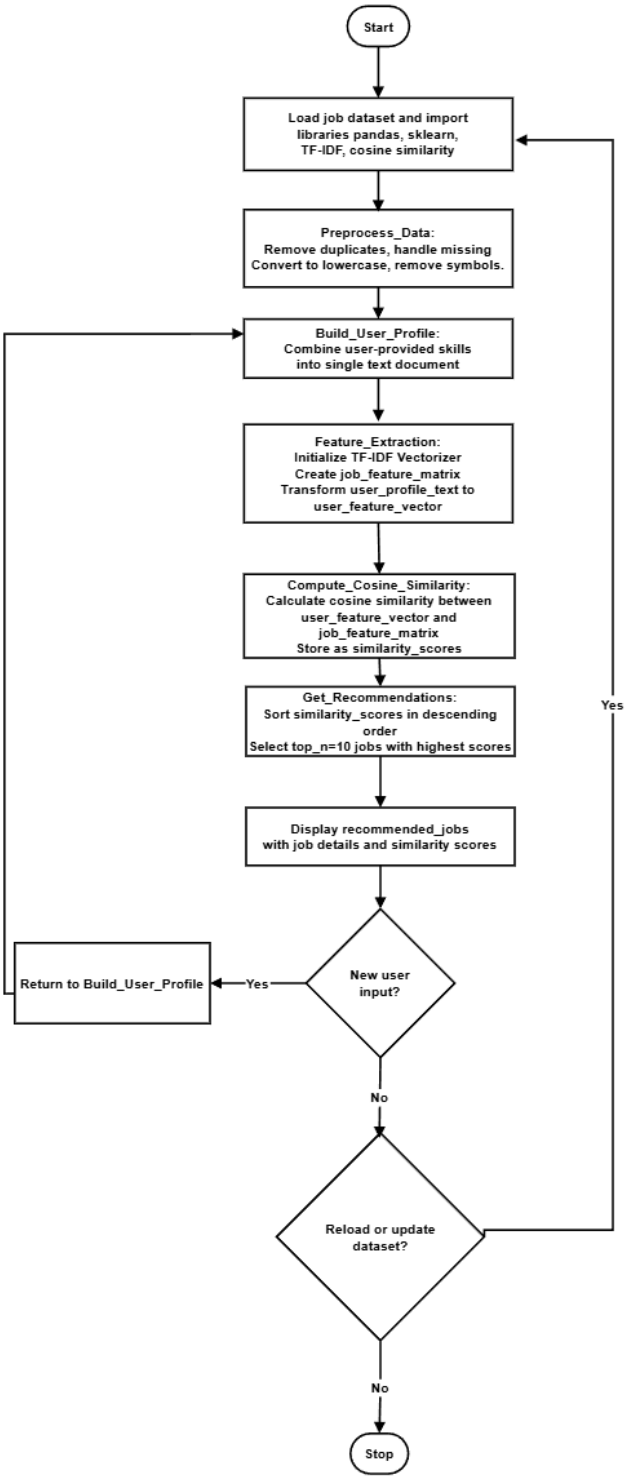


Figure 16: Content based filtering flowchart

4.5 Pseudocode of k-clustering

```
START

CLASS JobClusteringKMeans:

 FUNCTION Load_Data():

    IMPORT had (pandas, sklearn, K-Means) required libraries.

    LOAD the job dataset

    STORE dataset as jobs

    RETURN jobs

 FUNCTION Prepare_Features(jobs):

   IMPLEMENT  previously used TF-IDF job vectors taken as input features.

    RECORD vectors as job feature matrix

    RETURN job_feature_matrix

 FUNCTION Train_KMeans_Model(job_feature_matrix):

    INITIALIZE K-Means model

    SET number of clusters (k = 5)

    SET Random reproducibility state

    GENERATE K-Means model using job feature matrix .

    ADD cluster_labels to each job.

    RETURN cluster_labels

 FUNCTION Assign_Clusters(jobs, cluster labels):

    ADD cluster labels as new column in jobs dataset.

    RETURN updated_jobs

 FUNCTION view_Cluster_ Jobs( updated jobs cluster id):

    FILTER jobs of cluster cluster-id.
```

```
    SELECT job title, company name and job location.
    RETURN cluster_jobs
LOAD the dataset of the job with Load_Data()
RECORD the result as jobs
job_feature_matrix =
    Prepare_Features(jobs)
cluster_labels =
    Train_KMeans_Model(job_feature_matrix)
updated_jobs =
    Assign_Clusters(jobs, cluster_labels)
cluster_jobs =
    View Cluster Jobs( updated jobs, cluster id 0)
SHOW cluster_jobs
END
```

4.6 Flowchart of K-means clustering

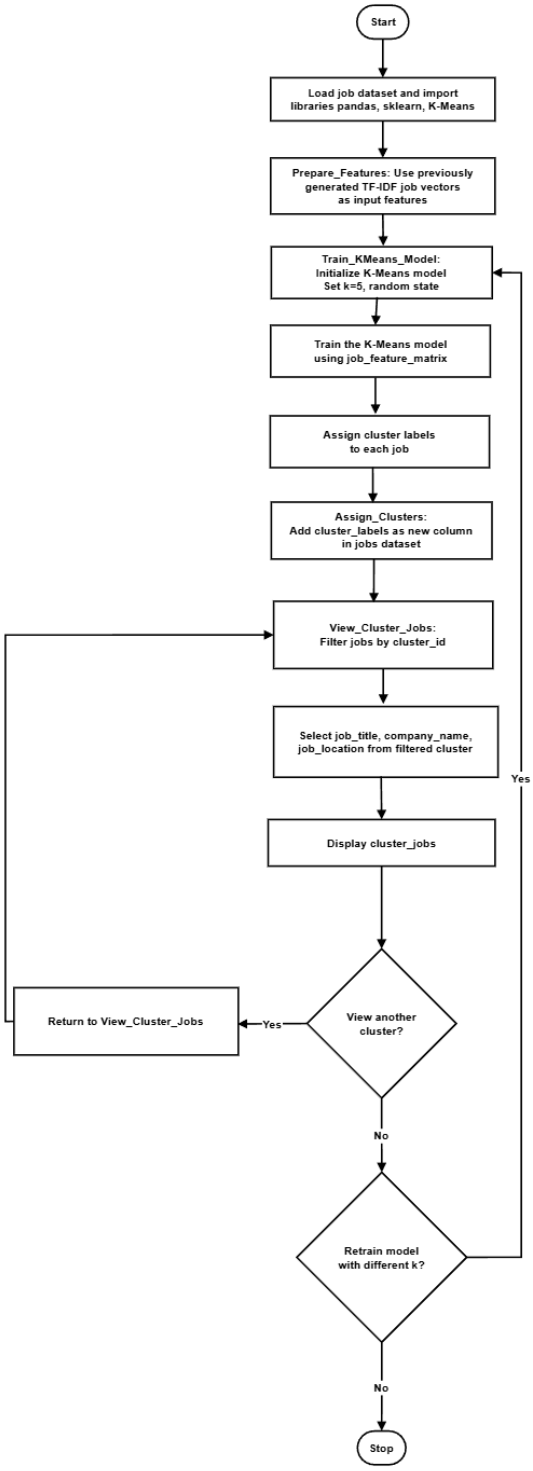


Figure 17: K-mean Clustering

5 Explanation of development process

5.1 Software used

5.1.1 Jupyter notebook

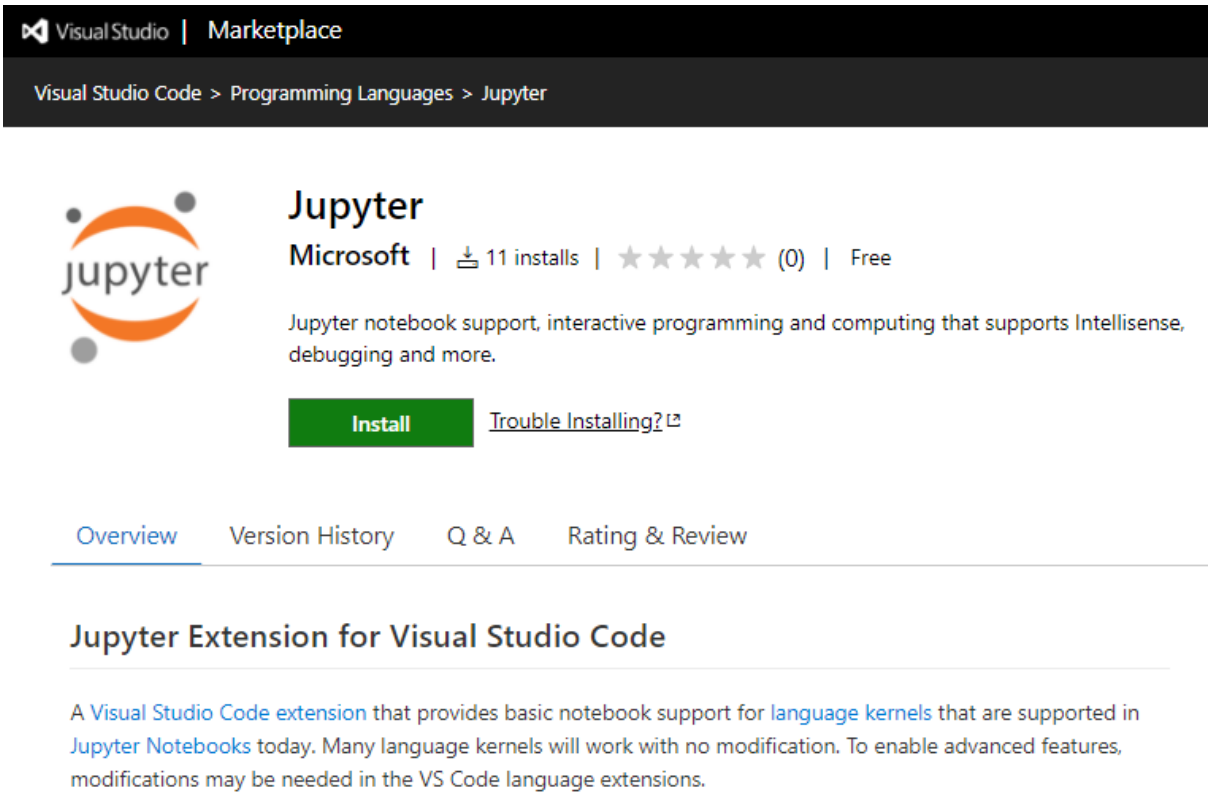


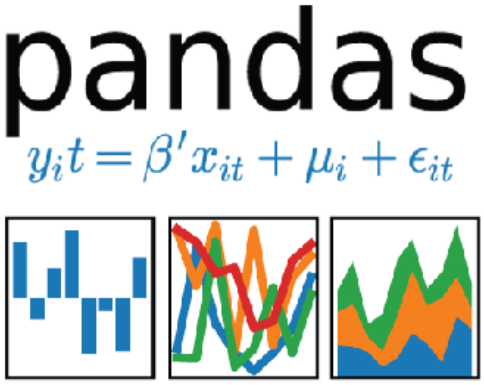
Figure 18: Jupyter

A Jupyter notebook is a document in which code is written and display its results in the same document. It is possible to run code, display results and to add explanations, formulas, and charts along with it. This simplifies work by making it understandable, clear, and repeatable. Jupyter Notebooks were commonly used in data science by global companies and organizations. They assist data scientists in analyzing data, experimenting with ideas, and disseminating findings in a streamlined manner.Jupyter notebooks are open source which is free to use. They are available on the Project Jupyter site or can be installed via the Anaconda toolkit of data science.

Even though Jupyter Notebooks can be used to support a wide range of programming languages, this description will concentrate on Python since it is the most popular in data science. Other languages are also supported including R, Julia, and Scala. (Pryke, 2025)

5.2 Libraries used

5.2.1 Pandas



	BandName	WavelengthMax	WavelengthMin
0	CoastalAerosol	450	430
1	Blue	510	450
2	Green	590	530
3	Red	670	640
4	NearInfrared	880	850
5	ShortWaveInfrared_1	1650	1570
6	ShortWaveInfrared_2	2290	2110
7	Cirrus	1380	1360

Figure 19: Pandas

pandas is a widely used open-source Python library that is used to perform data analysis and data manipulation. It is primarily intended to operate on tabular data (e.g., tables, spreadsheets, time-series data, etc.) and is intended to be used to load, clean, merge, and transform raw or messy data into structured formats, which can be analyzed with the help of other analytical tools.

The main program data structure in pandas is the DataFrame, a table with two dimensions whereby rows and columns symbolize records and variables respectively. Diverse data can be stored in a DataFrame such as numbers or text as well as categories; pandas is compatible with importing and exporting data in the form of CSV, Excel, and SQL databases. Most of its internal processes are coded in C or Cython to enhance its performance. Pandas is a widely used tool in data science and machine learning projects as well as data analysis due to its flexibility and ease of use. (NVIDIA, 2026)

5.2.2 NumPy

Figure 20: NumPy

NumPy is a library in Python that is designed to operate on arrays and numbers. It also assists in the calculations of such operations as linear algebra, matrices, and Fourier transforms. NumPy is an open-source library that can be free of charge and was developed by Travis Oliphant in 2005. NumPy is a name derived as Numerical Python. NumPy is more efficient than standard python lists in that the array object which is the array stores information on a continuous block of memory. This enables the computer to work on the data more successfully and this is very vital in data science where speed and performance are significant. NumPy is fast on contemporary processors (CPUs), and is programmed in Python, some of the (W3Schools.com, 2026)high-performance code written in C and C++. (W3Schools.com, 2026)

5.2.3 Matplotlib

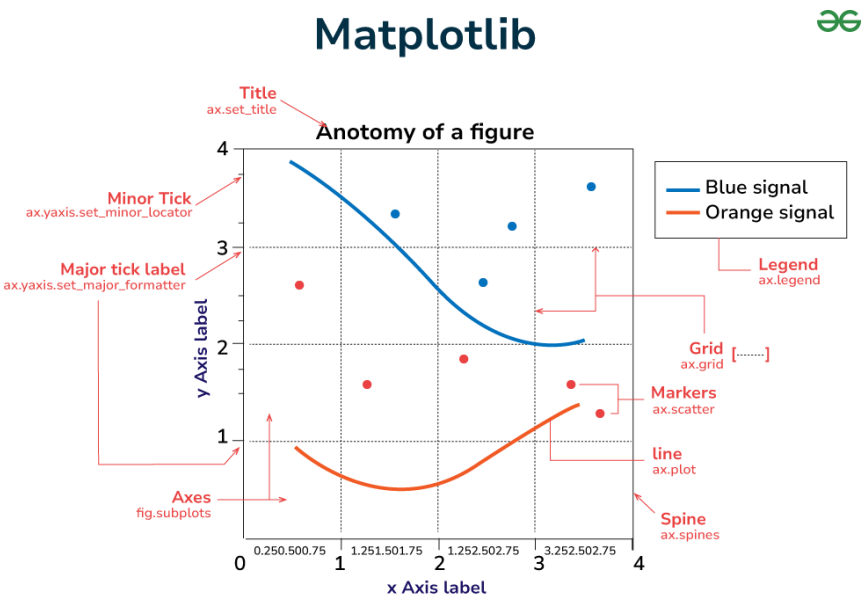


Figure 21: Matplotlib

Matplotlib is the Python library used the most frequently in data visualization. It is a cross platform library that assists in the creation of 2d plotting with data, which is stored in arrays. Matplotlib supports an object-oriented interface and thus is easy to add plots to applications written with Python GUI tools like PyQt, WxPython and Tkinter. It can be used in Jupyter Notebooks, web applications, and IPython shells and Python.

Matplotlib has been specifically created in order to produce effective and clear data visualizations. It is among the primary plotting libraries in Python and is easily applicable to beginners and advanced users. Matplotlib is a free open-source library and it can be used to create charts of various types, such as line plots, bar charts, scatter plots, histogram, pie charts, and area plots. Simple Python code can be used to create visual plots in Matplotlib only with a few lines of code, which is an additional advantage of the software (Tutorialspoint.com, 2025).

5.2.4 Seaborn

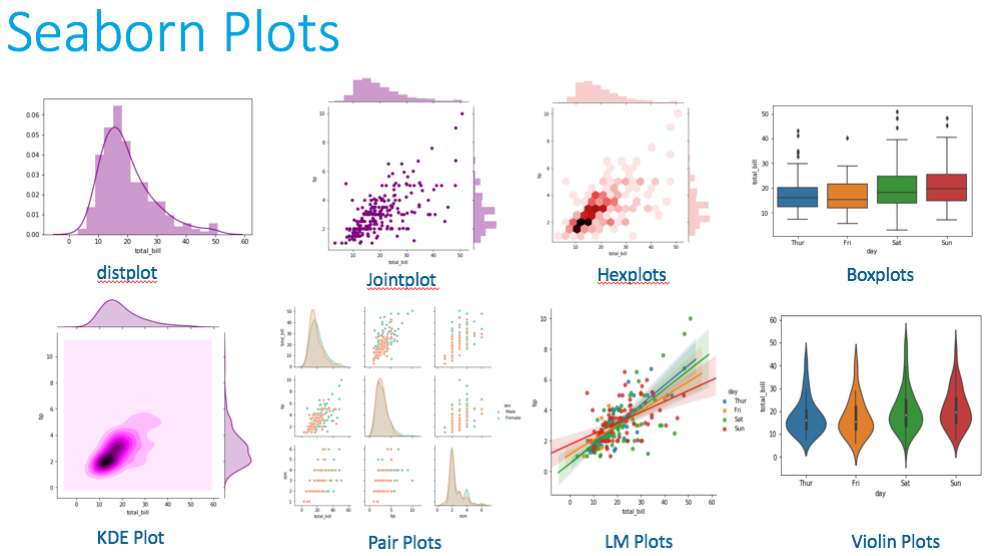


Figure 22:Seaborn

Seaborn is a Python data visualization library that enables one to create easy and clear and attractive statistical graphs. It can assist in the representation of information in a manner that is readable and well presented. Seaborn is developed over Matplotlib but it provides a more enhanced and easy to use interface to construct meaningful visualizations.Among the key strengths of Seaborn is the possibility to create complicated charts with just a few lines of code. It works with pandas DataFrames, and one can convert data to a visual format directly too. Seaborn also has inbuilt statistical capabilities including automatic addition of trend or regression lines to plots, which is useful to allow easy interpretation of patterns and insights in data by the user. (Gaurav, 2024)

5.2.5 Scikit-learn

Figure 23: sklearn

Scikit-learn (also known as sklearn) is a popular Python machine learning library that is open-source. It is simple and consistent with an interface that is easier to build machine learning, artificial intelligence, and statistical models.

Scikit-learn also has significant tools to classify, regress, cluster and reduce dimensions. It is developed over potent Python packages such as NumPY, SciPY and Matplotlib. With the provision of out of the box machine learning algorithms, scikit-learn eliminates the necessity to code complex models on your own, which typically needs a good background in mathematics. This is particularly helpful with beginners and enables users to be more concerned with data analysis and the acquisition of valuable insights. (Clark, 2025)

5.3 Components of library used

5.3.1 Cosine Similarity

Cosine similarity is a technique that is employed to quantify the similarity between two vectors. It operates in the search of the cosine of the angle between the vectors. When the angle is not large the vectors are more similar. Cosine similarity is applied in this project as the user skills are compared to job descriptions to identify the most relevant job suggestions.

5.3.2 TF-IDF Vectorizer

TF-IDF means Term Frequency -Inverse Document Frequency. One such tool is the TfidfVectorizer which comes as part of the scikit-learn library and is used to convert text data into a set of numerical feature vectors. It can assist in presenting job descriptions and skills in a format that can be interpreted by machine learning models.

5.3.3 Logistic Regression

Logistic Regression is a supervised machine learning model that is applied in classifying activities. This is applied in this project to classify job postings into the various job categories depending on text features extracted through TF-IDF.

5.3.4 Principal Component Analysis (PCA)

Principal Component Analysis (PCA) is a dimensionality reduction method which is applied to simplify data sets which are large. It compiles the features and preserves the most significant information. This project applies the PCA method to downsample TF-IDF vectors into two dimensions to be visualized.

5.3.5 K-Means Clustering

K-Means is a machine learning algorithm applied to unsupervised mode and which is used to cluster similar data points. The mechanism of operation is to assign the data to the closest cluster center. K-Means is applied in this project to cluster similar job ads in terms of their text characteristics.

5.4 Development Process

5.4.1 Importing Required Libraries

Import Required Libraries

In [1]:
```
# Import all necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')

# Set display options
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 100)
pd.set_option('display.width', None)

print(" All libraries imported successfully!")
```

```
All libraries imported successfully!
```

Figure 24: Importing Libraries

The development process began by the importation of all the required libraries of Python that I needed in handling data, machine learning, and visualization. All the libraries were chosen according to their purpose of assisting in various levels of system development.

- Pandas was employed to load, clean and manipulate the job dataset.
- Numerical operations and manipulation of arrays was done using NumPy.
- Data visualization and graph plotting were done using Matplotlib and Seaborn.

- Scikit-learn was also applied in performing machine learning algorithms including TF-IDF vectorization, Logistic Regression, K-Means clustering, PCA, and evaluation metrics.
- The unnecessary warning messages were blocked using warning library to ensure the output was clean.

The use of these libraries as imports meant that there would be all the necessary tools in order to process data, train and evaluate models and visualize the data in the most efficient way possible.

5.4.2 Data Loading

Dataset Loading

```
[2]: import pandas as pd

df = pd.read_csv("data_jobs.csv")
df.head()
```

Figure 25: Dataset Loading

Pandas library was used to load the dataset. A CSV file that had postings on the jobs was read in a DataFrame using the read csv function. The initial few records were presented to ensure that there was successful loading of data and to conceptualize the dataset structure.

5.4.3 Dataset Exploration

Data Exploration

[3]:

```
df.isnull().sum()
```

[3]:

```
job_title_short              0
job_title                    1
job_location              1045
job_via                      8
job_schedule_type        12667
job_work_from_home           0
search_location              0
job_posted_date              0
job_no_degree_mention        0
job_health_insurance         0
job_country                 49
salary_rate             752674
salary_year_avg         763738
salary_hour_avg         775079
company_name                18
job_skills              117037
job_type_skills         117037
dtype: int64
```

Figure 26: Data Exploration

Data exploration was performed to learn the data structure and quality of the data. The amount of the missing values in each column was determined by the isnull (). sum() function. This analysis has shown that many columns had some blank values especially those related to job location, salary details and job skills and therefore they require data cleanup prior to the development of the model.

5.4.4 Data cleaning and Text Preprocessing

Data Cleaning

Remove Missing Values

[4]:
```
df = df.dropna(subset=["job_title", "job_skills"])
df.shape
```

[4]: (668784, 17)

Select Relevant Columns

[5]:
```
df = df[
    [
        "job_title_short",
        "job_title",
        "job_location",
        "job_work_from_home",
        "company_name",
        "job_skills",
        "salary_year_avg"
    ]
]

df.head()
```

[5]:

	job_title_short	job_title	job_location	job_work_from_home	company_name	job_skills	salary_year_avg
1	Data Analyst	Data Analyst	Guadalajara, Jalisco, Mexico	False	Hewlett Packard Enterprise	['r', 'python', 'sql', 'nosql', 'power bi', 't...	NaN
2	Data Engineer	Data Engineer/Scientist/Analyst, Mid or Senior...	Berlin, Germany	False	ALPHA Augmented Services	['python', 'sql', 'c#', 'azure', 'airflow', 'd...	NaN
3	Data Engineer	LEAD ENGINEER · PRINCIPAL ANALYST · PRINCIPAL ...	San Antonio, TX	False	Southwest Research Institute	['python', 'c++', 'java', 'matlab', 'aws', 'ts...	NaN
4	Data Engineer	Data Engineer · Sr Jobs	Washington, DC	False	Kristina Daniel	['bash', 'python', 'oracle', 'aws', 'ansible',...	NaN
5	Data Engineer	GCP Data Engineer	Anywhere	True	smart folks inc	['python', 'sql', 'gcp']	NaN

After Select Relevant Columns

[6]:
```
df.shape
```

[6]: (668784, 7)

Remove Duplicate Records

[7]:
```
df = df.drop_duplicates()
df.head()
```

[7]:

	job_title_short	job_title	job_location	job_work_from_home	company_name	job_skills	salary_year_avg
1	Data Analyst	Data Analyst	Guadalajara, Jalisco, Mexico	False	Hewlett Packard Enterprise	['r', 'python', 'sql', 'nosql', 'power bi', 't...	NaN
2	Data Engineer	Data Engineer/Scientist/Analyst, Mid or Senior...	Berlin, Germany	False	ALPHA Augmented Services	['python', 'sql', 'c#', 'azure', 'airflow', 'd...	NaN
3	Data Engineer	LEAD ENGINEER · PRINCIPAL ANALYST · PRINCIPAL ...	San Antonio, TX	False	Southwest Research Institute	['python', 'c++', 'java', 'matlab', 'aws', 'ts...	NaN
4	Data Engineer	Data Engineer · Sr Jobs	Washington, DC	False	Kristina Daniel	['bash', 'python', 'oracle', 'aws', 'ansible',...	NaN
5	Data Engineer	GCP Data Engineer	Anywhere	True	smart folks inc	['python', 'sql', 'gcp']	NaN

Text Preprocessing

Clean Text Columns

[8]:
```
df["job_title"] = df["job_title"].str.lower()
df["job_skills"] = df["job_skills"].astype(str).str.lower()
```

Create Combined Text Feature

[9]:
```
df["job_text"] = df["job_title"] + " " + df["job_skills"]
```

Save Cleaned Dataset

[10]:
```
df.to_csv("cleaned_jobs.csv", index=False)
```

Figure 27: Data Cleaning and pre processing

Data cleaning was done after data exploration to enhance the quality of data and make it suitable to be used in machine learning models. The records that had blank values in the key record fields like job title and job skills were dropped since the two are vital in job recommendation and text analysis. This was done to make sure that only significant and complete job advertisements were used in the system.Then, the selection was made of only the relevant columns that were covered under job information; such as job title, job location, company name, work-from-home status, work needs, and salary information. The chosen features allowed to minimize the complexity of datasets and enhance the processing efficiency. During model training, duplicate job records were also eliminated to eliminate redundancy and biasness.Preprocessing of text was then used in order to prepare textual data to be processed by the natural language processing. Job titles and job skills were all changed into lowercase in order to be consistent and not to consider the same words as different tokens. The job title and job skills column were merged to form one text feature, which made the system to record more contextual information about job description.

Lastly, the processed and cleaned data were stored to be used later. To minimize the computational cost in training the model a random sample of the dataset was chosen. This trained data was applied in the job recommendation system to extract features and build models.

5.4.5 AI model Training

AI Model Training

```
[11]: import pandas as pd

df = pd.read_csv("cleaned_jobs.csv")

# Use sampling so models run fast
df_sample = df.sample(20000, random_state=42)

df_sample.shape

[11]: (20000, 8)
```

Figure 28: Importing Dataset After Cleaning

Once the cleaned dataset was ready the next step was to train AI models to recommend jobs and analyze them. The filtered data was reload and a random sample of the records was picked to minimize the cost of computing and enhance the speed of processing. The sampling was made such that the models could be trained effectively and at the same time have representative

job data.Implementation of various AI techniques such as content-based filtering, NLP-based classification, and clustering was then performed on the sampled dataset.

5.4.6 Content-Based job recommendation

Content-Based Filtering (TF-IDF + Cosine Similarity)

This section implements a content-based job recommendation system using TF-IDF vectorization and cosine similarity to recommend jobs based on user-provided skills.

[12]:
```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# TF-IDF Vectorization
tfidf = TfidfVectorizer(stop_words="english", max_features=3000)
job_vectors = tfidf.fit_transform(df_sample["job_text"])

# User input
user_input = "python sql data analysis machine learning"
user_vector = tfidf.transform([user_input])

# Cosine similarity
similarity_scores = cosine_similarity(user_vector, job_vectors).flatten()

# Top 5 job recommendations with similarity score
top_indices = similarity_scores.argsort()[-5:][::-1]

top_jobs = df_sample.iloc[top_indices].copy()
top_jobs["similarity_score"] = similarity_scores[top_indices]

top_jobs[["job_title", "company_name", "job_location", "similarity_score"]]
```

[12]:

	job_title	company_name	job_location	similarity_score
89274	machine learning engineer	TEKsystems (Allegis Group Singapore Pte Ltd)	Singapore	0.726809
426146	machine learning engineer	SQLink Group	Tel Aviv-Yafo, Israel	0.709598
226648	machine learning engineer	TikTok	San José Province, San José, Costa Rica	0.709598
212373	machine learning engineer	Why Hiring	Anywhere	0.709598
14777	machine learning engineer	Turing	Anywhere	0.709598

Figure 29: Content-Based Filtering

The content-based filtering was also applied to suggest jobs using the similarity between the job descriptions and skills provided by users. This practice aims at aligning the information in the job ads with the talent of the user as opposed to the preferences of the other users.

TF-IDF Vectorization

The textual job information was converted into numerical feature vectors using TF-IDF (Term Frequency -Inverse Document Frequency). It places more emphasis on those words that are common in a job description but uncommon to the whole dataset.

Term Frequency (TF) is calculated as:

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

Inverse Document Frequency (IDF) is calculated as:

$$IDF(t) = \log\left(\frac{N}{DF(t)}\right)$$

Figure 30:Tf formula

where:

$\boldsymbol{N}$ is the total number of documents

$DF(t)$ is the number of documents containing term

$$TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$$

Figure 31: TD-IDF Score calculation

The process converts job titles and skills into numerical vectors which depict the significance of each term.

Cosine Similarity

Cosine similarity was applied to evaluate the similarity between the user skill and job vectors after the process of vectorization. Cosine similarity computes the angle between two vectors and it shows how closely the two vectors are associated.

The formula of the cosine similarity is:

$$\text{Cosine Similarity}(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

Figure 32:Cosine Similarity

where:

$\boldsymbol{A}$: is the user skill vector

$\boldsymbol{B}$: represents the job description vector.

$\boldsymbol{A} \cdot \boldsymbol{B}$ denotes the dot product of the vectors.

$\|A\|$ and $\|B\|$

M magnitudes of vectors are called B.

A lower score would be more similar whereas a more similar score would be a cosine similarity score more likely to be 1.

The system calculates similarity scores of cosine among the user skill vector and job vectors. The most similar jobs are chosen and suggested to the user. This is to make sure that jobs suggested are very much aligned to the skills and interests of the user.

5.4.7 NLP-Based job Category Classification

NLP Classification Model (Logistic Regression)

```
[30]: # Import libraries for splitting data and training the model
      from sklearn.model_selection import train_test_split
      from sklearn.linear_model import LogisticRegression

      # Set input data and job labels
      X = job_vectors
      y = df_sample["job_title_short"]

      # Divide data into training and testing parts
      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, random_state=42
      )

      # Create and train the Logistic Regression model
      clf = LogisticRegression(max_iter=1000)
      clf.fit(X_train, y_train)

[30]: LogisticRegression
      LogisticRegression(max_iter=1000)

[14]: # User-provided skills entered as a single text string
      user_skills = "python sql machine learning data analysis"

      # Convert the user skills text into a TF-IDF vector
      # using the same TF-IDF model trained on job descriptions
      user_vector = tfidf.transform([user_skills])

[15]: # Predict the most suitable job role based on the user's skill vector
      predicted_job_role = clf.predict(user_vector)
      predicted_job_role

[15]: array(['Machine Learning Engineer'], dtype=object)

[16]: # Get probability scores for each job category
      probs = clf.predict_proba(user_vector)

      # Get all job category names
      classes = clf.classes_

      # Store job categories and their probabilities in a table
      prediction_df = pd.DataFrame({
          "Job Category": classes,
          "Probability": probs[0]
      }).sort_values(by="Probability", ascending=False)

      # Store highest probability as NLP model score
      nlp_score = prediction_df["Probability"].max()

      # Explicitly display the recommendation table
```

Figure 33: NLP classification

A classification model based on NLP and Logistic Regression was adopted to obtain the most appropriate job role depending on the skills that the user offers. This is in contrast to content-based filtering because it is a supervised learning technique that learns using labeled job data to categorize user skills into the predefined job categories.

TF-IDF Feature Representation

Before the use of Logistic Regression, the textual job contents were transformed into numeric feature vectors via TF-IDF (Term Frequency -Inverse Document Frequency). This transformation guarantees that the job descriptions are reflected as well as the skills provided by the users in a shared numerical space.

The TF, IDF and TF-IDF computations are performed in the same manner as those of the content-based filtering section:

- TF (Term Frequency)
- IDF (Inverse Document Frequency)
- TF-IDF Score

Such vectors are the input of the classification model.

Logistic Regression Classification

Logistic Regression is a linear combination of the TF-IDF features to obtain the probability of each job category.

$$z_k = w_k \cdot x + b_k$$

Figure 34: Logistic Regression Linear Function

Where:

x is the TF-IDF feature vector

w_k is the weight vector of job category k

b_k is the bias term

Given the presence of several job categories in the problem, the model uses the softmax function to change the linear scores to probabilities.

$$P(y = k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}$$

Figure 35: Softmax Probability Function

Job Role Prediction

The job role that has been predicted is the one that scores highest in terms of probability.

$$\hat{y} = \arg\max_{k} P(y = k \mid x)$$

Figure 36: Job role Prediction rule

This makes the system to suggest the job role that is most appropriate to the user based on the patterns that are learned in the dataset.

Probability Estimation and NLP Model Score

Probability values of all job categories are also calculated using the model. These are the chances of the prediction in each role. The largest probability value is chosen to be the NLP model score which is the confidence level of the final prediction.

Result Interpretation

The Logistic Regression model gives one estimated job position and probability measurements of all the classes. The method allows the job role to be predicted with accurateness and interpretability thus effective in the classification-based job recommendation within the system.

5.4.8 Job clustering using K-means

K-Means Clustering Model

[17]:
```
from sklearn.cluster import KMeans

# Train K-Means model
kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(job_vectors)

# Assign cluster labels
df_sample["cluster"] = clusters

# View jobs from one cluster
df_sample[df_sample["cluster"] == 0][
    ["job_title", "company_name", "job_location"]
].head()
```

[17]:

	job_title	company_name	job_location
419419	data system analyst with sql	Mindteck	Beloit, WI
536331	senior platform engineer	Manulife	Toronto, ON, Canada
482425	stage data engineer f/h	Sonergia	Marseille, France
352087	data engineer - paris (f/h)	Stack Labs	Paris, France
192318	big data engineer	Indocyber Global Teknologi, PT	Jakarta, Indonesia

Figure 37: K-mean Clustering

The K-Means model of clustering was used to cluster similar job postings in terms of their textual information. It is an unsupervised learning method that is, it does not operate with predefined job labels but rather it attempts to recognize natural patterns in the data itself.

The TF-IDF vectorization was initially used to convert job titles and skills into numerical feature vectors. These vectors are values of the significance of terms in each job description and they result in the input of the clustering operation.

K-Means Clustering Process

The K-Means algorithm divides the job vectors into a specified group of clusters(k=5). A cluster denotes a group of job openings having common skills and job description.

$$\min \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|^2$$

Figure 38:K-means Objective Function

Where:

xi: **represents** a job vector

μc: represents the centroid of cluster

n: n is the total number of job postings

The algorithm then repeats allotment of jobs to the closest centroid and makes the centroid positions until convergence.

Once the model has been trained, a cluster label is provided to each job posting. The same cluster jobs have similar characteristics e.g. necessary skills or job description. Jobs that were sampled in a chosen cluster are shown to demonstrate the grouping effect.

Result Interpretation

K-Means clustering model is effective in terms of grouping similar job ads, including data engineering and analytics, into identical clusters. Even though it does not directly suggest a job role it offers valuable information about job groupings and aids in exploration analysis as part of the job recommendation system.

5.4.9 Cluster Visualization Using PCA

Cluster Visualization

```
[29]: # Import PCA to reduce data dimensions
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Convert job vectors into two dimensions for plotting
pca = PCA(n_components=2, random_state=42)
reduced_vectors = pca.fit_transform(job_vectors.toarray())

# Plot the job clusters using a scatter plot
plt.figure(figsize=(8, 6))
plt.scatter(
    reduced_vectors[:, 0],
    reduced_vectors[:, 1],
    c=df_sample["cluster"],
    s=10
)

# Add chart title and labels
plt.title("K-Means Clustering of Job Postings (PCA Visualization)")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

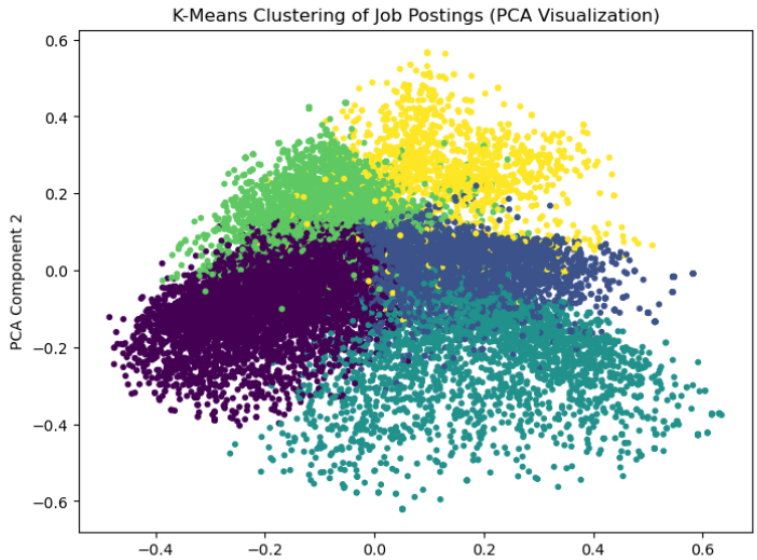


Figure 39: Cluster Visualization

To visualise the clusters of the K-Means algorithm, Principal Component Analysis (PCA) was used to decrease high-dimensional TF-IDF job vectors in two dimensions. PCA retains the greatest significance of the data and also allows it to be represented graphically.The reduced vectors were represented in a scatter plot whereby each point represents a job posting and colors are used to represent membership in a cluster. It is indicated in the visualization that the job openings with comparable features are clustered tightly, which proves the successful K-Means clustering method.

5.5 Model Evaluation and comparison

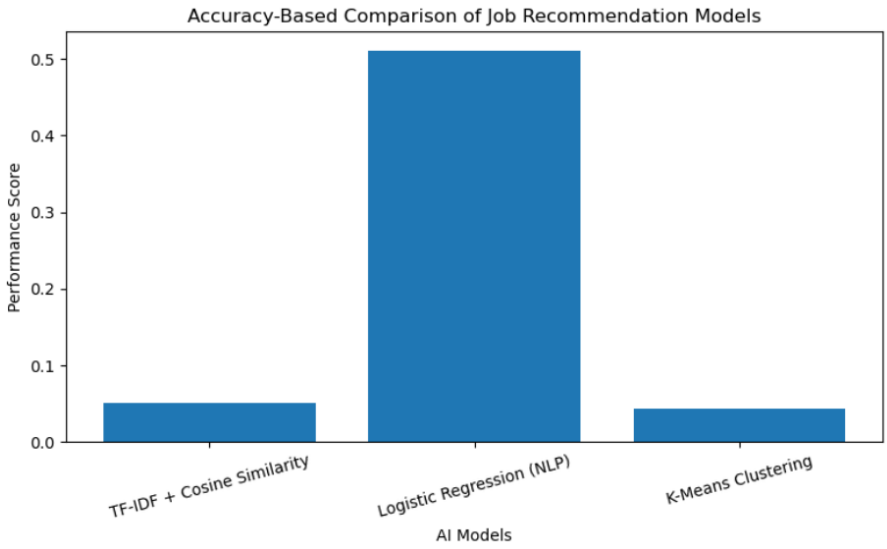


Figure 40: AI model comparison

The three AI models have been tested based on task-specific metrics instead of one metric of accuracy. The cosine similarity model of TF-IDF was tested in terms of the average similarity score that indicates the degree of job recommendation matching the skills of users. The NLP classification model was determined according to the greatest prediction likelihood of the job role which is the confidence of the job role being predicted. The silhouette score was used to assess the quality of the job groupings when the K-Means clustering model was used.

The performance of the models was compared using Bar chart. The findings show that the two models have their own contribution to the job recommendation system and they do not compete against each other based on one evaluation criterion.

5.5.1 Comparison of AI Models used In Job Recommendation System

Table 1: Comparison of AI Models used In Job Recommendation System

AI module	Technique	Purpose	output	Evaluation Metric	Result
Content-Based Recommendation	TF-IDF + Cosine Similarity	Recommend jobs matching user skills	Top-5job recommendations	Hit@5, Avg similarity	Hit@5 = 1.00, Avg similarity ≈ 0.05
NLP Classification Model	Logistic Regression	Predict most suitable job role	Single job category	Top-1 accuracy, Probability	Accuracy=96%, Max prob ≈ 0.51
Clustering Model	K-Means	Group similar job postings	Job cluster & dominant role	Silhouette score, Cluster accuracy	Silhouette≈0.04, Accuracy ≈ 68%

5.5.2 Comparison of Implemented AI Models

Table 2: Comparison of Implemented AI Models

AI module	Technique	Input	output	Evaluation Metric	Result
Content-Based Recommendation	TF-IDF + Cosine Similarity	User skill text + job descriptions	Top-5 similar job postings with similarity scores	Average cosine similarity	Avg similarity ≈ 0.0516
NLP Classification Model	Logistic Regression (TF-IDF features)	User skill text	Predicted job category	Maximum prediction probability	Max probability ≈ 0.51
Clustering Model	K-Means + PCA visualization	Job description vectors	Job cluster & dominant role	Silhouette score,	Silhouette ≈ 0.043

6 Conclusion

6.1 Analysis of the work done

This document is the Milestone 1 of the AI Coursework 2 that is necessary to complete 75% of the AI module. The AI-based Job Recommendation System has been studied and analyzed in detail in this document. An extensive study of job recommendation systems and the filtering and machine learning algorithm most appropriate in job recommending based on user skills and job descriptions has been done.

Besides this, short research on the functioning of the real-world job recommendation systems, and the methods that are widely employed in the large-scale systems of the industry in matching jobs has been carried out as an auxiliary case study. The activities of applying three various AI methods, i.e. content-based filtering based on TF-IDF and cosine similarity, classification based on Logistic Regression and unsupervised learning implemented with the help of K-Means clustering applied to the job recommendation system have been successfully fulfilled.The pseudocode and flowcharts of the three algorithms were also designed and written along with the implementation to include the system logic and the flow in a clear way. Hence, the given document reflects the analysis of the work that has already been done regarding the AI Job Recommendation System.

.

6.2 How this addresses real world problems

In real world job seeker can be easily frustrated while searching for appropriate jobs because of the numerous job ads and inability to understand the skills needed. This results in a lack of efficient job seeking and job-skills matching.

The proposed AI-based Job Recommendation System will resolve this issue by examining the user skills and job descriptions based on the method of machine learning. Distributing jobs based on their content and usage should be assisted by content-based filtering and classification to give a personalized and correct job suggestion and clustering slightly similar jobs to explore them easier. Comprehensively, the system minimizes the manual work, increases the accuracy of matching, and facilitates enhanced career decision-making.

6.3 Further work

A number of works can be included in the job recommendation system to improve the quality of services. As an example, it would be more efficient to have user feedback where the user can rate or rate the job recommendations they get. In this way, the system will be able to constantly learn about the interaction with users and give more precise and tailored job recommendations.

Moreover, more advanced natural language processing methods, including semantic embeddings and skill-level extraction might be added to learn more about job demands and profile of users. It is also possible to include location preference, salary expectations, and experience level to suggest jobs that are more aligned to user goals. Therefore, such improvements are a way of future working to increase the effectiveness and practicality of job recommendation system.

7 References

Alfaifi, Y. H., 2024. *Recommender Systems Applications: Data Sources, Features, and Challenges.* [Online] Available at: https://www.mdpi.com/2078-2489/15/10/660 [Accessed 10 12 2025].

Anon., n.d. [Online].

Anon., n.d. s.l., s.n.

Clark, B., 2025. *What is Scikit-Learn (Sklearn)?.* [Online] Available at: https://www.ibm.com/think/topics/scikit-learn [Accessed 6 1 2026].

Crabtree, M., 2023. *What is Natural Language Processing (NLP)? A Comprehensive Guide for Beginners.* [Online] Available at: https://www.datacamp.com/blog/what-is-natural-language-processing
[Accessed 14 12 2025].

Crabtree, M., 2023. *What is Natural Language Processing (NLP)? A Comprehensive Guide for Beginners.* [Online] Available at: https://www.datacamp.com/blog/what-is-natural-language-processing
[Accessed 14 12 2025].

Craig, L., Laskowski, N. & Tucci, L., 2024. *What is AI (Artificial Intelligence)? Definition, Types, Examples & Use Cases.* [Online] Available at: https://www.techtarget.com/searchenterpriseai/definition/AI-Artificial-Intelligence
[Accessed 14 12 2025].

databricks, 2023. *what is dataset?.* [Online] Available at: https://www.databricks.com/glossary/what-is-dataset [Accessed 11 12 2025].

Gadegaonkar, S., Lakhwani, D., Marwaha, S. & Salunke, P. A., 2023. *Job Recommendation System using Machine Learning.* Coimbatore, India, IEEE.

Gaurav, S., 2024. *Introduction to Seaborn Python Library - Scaler Topics.* [Online]
Available at: https://www.scaler.com/topics/seaborn-in-python/ [Accessed 6 1 2026].

GeeksforGeeks, 2018. *What is Machine Learning?.* [Online] Available at: https://www.geeksforgeeks.org/machine-learning/ml-machine-learning/
[Accessed 14 12 2025].

Haldar, A., 2024. *Job Recommendation System using LinkedIn User Profiles.* Vellore, India, IEEE.

Kavlakoglu, E., 2024. *What is content-based filtering?.* [Online] Available at: https://www.ibm.com/think/topics/content-based-filtering [Accessed 9 12 2025].

Kavlakoglu, E. & Winland, V., 2024. *What is k-means clustering?.* [Online] Available at: https://www.ibm.com/think/topics/k-means-clustering [Accessed 14 12 2025].

Kumari, J., 2024. *How Do You Convert Text Documents to a TF-IDF Matrix with tfidfvectorizer?.* [Online]
Available at: https://www.analyticsvidhya.com/blog/2024/07/tf-idf-matrix/ [Accessed 16 12 1025].

Kumar, S., 2021. *What is Recommdation System?.* [Online] Available at: https://www.analyticsvidhya.com/blog/2021/07/recommendation-system-understanding-the-basic-concepts/
[Accessed 10 12 2025].

Mhamdi, D. et al., 2020. *Job Recommendation based on Job Profile Clustering and Job Seeker Behavior.* [Online] Available at: https://www.sciencedirect.com/science/article/pii/S1877050920318020 [Accessed 13 12 2025].

Nikolopoulou., K., 2023. *What Is Data Mining? | Definition & Techniques.* [Online] Available at: https://www.scribbr.com/ai-tools/data-mining/ [Accessed 14 12 2025].

NVIDIA Data Science Glossary, 2025. *K-Means Clustering Algorithm.* [Online] Available at: https://www.nvidia.com/en-eu/glossary/k-means/ [Accessed 16 12 2025].

NVIDIA, 2026. *What Is Pandas?.* [Online] Available at: https://www.nvidia.com/en-us/glossary/pandas-python/ [Accessed 6 1 2026].

Ph.D, J. M., 2024. *what is Collaborative Filtering?.* [Online] Available at: https://www.ibm.com/think/topics/collaborative-filtering [Accessed 11 12 2025].

Pryke, B., 2025. *How to Use Jupyter Notebook: A Beginner's Tutorial – Dataquest.* [Online] Available at: https://www.dataquest.io/blog/jupyter-notebook-tutorial/ [Accessed 6 1 2026].

Saini, A. & Tripathi, J., 2024. *Job recommendation system based on content-based filtering.* [Online] Available at: https://www.taylorfrancis.com/chapters/edit/10.1201/9781032644752-6/job-recommendation-system-based-content-based-filtering-anu-saini-jyoti-tripathi-mohd-faraz-mohd-kaif [Accessed 11 12 2025].

Sheldon, R., 2023. *defination pseudocode.* [Online] Available at: https://www.techtarget.com/whatis/definition/pseudocode [Accessed 13 12 2025].

Stratascratch.com, 2023. *Step-by-Step Guide to Building Content-Based Filtering - StrataScratch.* [Online] Available at: https://www.stratascratch.com/blog/step-by-step-guide-to-building-content-based-filtering/
[Accessed 14 12 2025].

Tutorialspoint.com, 2025. *Matplotlib Tutorial.* [Online] Available at: https://www.tutorialspoint.com/matplotlib/index.htm [Accessed 6 1 2026].

W3Schools.com, 2026. *NumPy Introduction.* [Online] Available at: https://www.w3schools.com/python/numpy/numpy_intro.asp [Accessed 6 1 2026].